

RAPPORT DE CADRAGE & SPÉCIFICATIONS

CAHIER DES CHARGES

**Webpick Initializr : Générateur Dynamique de Projets et
d'Infrastructure CI/CD**

Réalisé par :

BOUANANI Abderrahman
Étudiant Ingénieur Stagiaire
École Nationale des Sciences Appliquées

Sous la supervision de :

M. Abderrahim GOUNANE
Maître de Stage / Chef de projet
Agence Webpick

Table des matières

1	Introduction et Contexte du Projet	2
1.1	Présentation de l'Entreprise Webpick	2
1.2	Objectifs du Stage d'Ingénierie	2
1.3	Problématique : L'initialisation répétitive (Day 0)	2
2	Analyse du Besoin et État de l'Art	2
2.1	Besoins Fonctionnels Principaux	2
2.2	État de l'Art et Comparatif Technique	2
3	Conception et Spécifications Techniques	3
3.1	Stack Technologique Retenue	3
3.2	Architecture Applicative (Clean Architecture)	3
3.3	Isolation des Responsabilités	4
4	Planification et Livrables du Stage	4
4.1	Planning Prévisionnel	5
4.2	Critères d'Acceptation	5
5	Conclusion	6

1 Introduction et Contexte du Projet

1.1 Présentation de l'Entreprise Webpick

Webpick est une agence digitale moderne, spécialisée dans le développement d'applications Web et Mobiles à forte valeur ajoutée, ainsi que dans le conseil et l'intégration de solutions Cloud et DevOps. Pour maintenir un niveau élevé de compétitivité, l'agence rationalise en permanence ses processus de livraison logicielle.

1.2 Objectifs du Stage d'Ingénierie

Ce stage d'été 2026, réalisé par un élève-ingénieur de l'ENSA Agadir, s'inscrit dans une démarche de recherche, d'optimisation et d'industrialisation des méthodes de développement de l'agence. L'objectif est double :

- **Académique** : Mettre en pratique les concepts avancés d'architecture logicielle (Clean Architecture, programmation orientée objet), de validation de modèles, et de gestion de flux asynchrones.
- **Professionnel** : Livrer un produit fonctionnel, robuste et immédiatement exploitable par les équipes techniques de Webpick pour accélérer la phase d'initialisation de projet (Day 0).

1.3 Problématique : L'initialisation répétitive (Day 0)

La mise en place de l'écosystème technique d'un nouveau projet implique le démarrage de tâches répétitives à faible valeur ajoutée : initialisation de la structure MVC, configuration des descripteurs de dépendances, écriture des Dockerfiles multi-stage, et rédaction des pipelines CI/CD Jenkins. Cette étape manuelle est sujette aux erreurs, ralentit le démarrage effectif et nuit à l'homogénéité du parc applicatif.

2 Analyse du Besoin et État de l'Art

2.1 Besoins Fonctionnels Principaux

La plateforme **Webpick Initializr** doit répondre aux exigences suivantes :

1. **Interface Graphique Intuitive** : Un portail web unique permettant de paramétrer les attributs du projet cible (nom, package, BDD, options de déploiement).
2. **Moteur de Remplacement Dynamique** : Un module de templating capable d'injecter les variables utilisateurs dans des structures de fichiers prédéfinies.
3. **Export Automatisé** : Téléchargement direct d'une archive compressée ZIP contenant le projet prêt à compiler et à être exécuté.
4. **Intégration DevOps Native** : Fournir automatiquement les descripteurs d'environnement (Docker Compose) et d'automatisation CI/CD (Jenkinsfile).

2.2 État de l'Art et Comparatif Technique

Le tableau suivant résume le positionnement de notre solution vis-à-vis des outils du marché :

Solution	Limites identifiées	Valeur ajoutée de Webpick Initializr
Spring Initializr	Restreint à l'écosystème Java/Spring Boot. N'intègre pas la partie Frontend Angular ni Jenkins.	Approche Fullstack combinant Angular et Spring Boot adaptés aux conventions de l'agence.
Yeoman (CLI)	Nécessite l'installation d'outils Node en local. Courbe d'apprentissage CLI pour les profils non-JS.	Interface Web ergonomique hébergée en interne, accessible sans configuration locale préalable.
Spotify Backstage	Solution extrêmement complexe nécessitant une infrastructure et une maintenance lourdes.	Application agile, légère et ciblée, conçue spécifiquement pour le flux de travail Webpick.

TABLE 1 – Positionnement technologique du projet

3 Conception et Spécifications Techniques

Le projet exige une séparation stricte des préoccupations (Separation of Concerns). Le choix s'est ainsi porté sur une architecture découplée.

3.1 Stack Technologique Retenue

Afin de garantir un rendu de niveau professionnel, la stack technique s'appuie sur des frameworks d'entreprise rigoureux :

- **Frontend — Angular 18+ :** Choisi pour son architecture orientée composants, son typage fort en TypeScript, l'utilisation de **RxJS** et des **Reactive Forms** (standard de choix pour la gestion robuste de formulaires conditionnels complexes).
- **Backend — Spring Boot 3+ (Java 21) :** Standard industriel pour le développement d'API sécurisées et robustes. Il tirera parti des fonctionnalités modernes de Java 21 (Records, pattern matching).
- **Moteur de Templating — Apache FreeMarker :** Utilisé côté backend pour compiler dynamiquement les fichiers sources à partir de modèles prédéfinis.
- **Stockage — PostgreSQL :** Base de données relationnelle pour le suivi des configurations générées.
- **Conteneurisation & CI/CD — Docker, Docker Compose & Jenkins :** Docker et Compose pour assurer la conteneurisation et la portabilité locale de la solution. Jenkins est intégré pour héberger et orchestrer les pipelines CI/CD automatisés qui seront générés dynamiquement.

3.2 Architecture Applicative (Clean Architecture)

Le backend Spring Boot sera découpé en quatre couches concentriques afin d'assurer l'indépendance totale de la logique métier vis-à-vis des détails technologiques et des frameworks extérieurs :

1. **Couche Domaine (Entities)** : Couche centrale contenant la pure logique métier, les modèles abstraits de projets à générer et les règles de validation fondamentales. Elle est totalement indépendante des frameworks, bases de données ou bibliothèques tierces.
2. **Couche Présentation (API REST)** : Expose les points d'accès (Endpoints) HTTP et assure la validation préliminaire des requêtes ('@Valid').
3. **Couche Application (Cas d'utilisation)** : Contient la logique d'orchestration et les scénarios d'utilisation (validation métier, appel au moteur de templating, archivage ZIP).
4. **Couche Infrastructure** : Gère les détails techniques et physiques (accès au système de fichiers pour lire les modèles, persistance PostgreSQL, intégrations d'API Git).

3.3 Isolation des Responsabilités

Afin de garantir un périmètre d'action strictement délimité et maîtrisable, l'application se concentre exclusivement sur l'orchestration et la gestion de la génération de code (le domaine métier de Webpick Initializr). Toutes les responsabilités et modules n'appartenant pas directement à ce flux de génération (comme la gestion de l'infrastructure d'hébergement finale ou la surveillance des pipelines à chaud) sont exclus ou découplés de la logique principale de l'application via des abstractions et interfaces.

4 Planification et Livrables du Stage

Le stage est planifié sur une durée de 8 semaines avec les jalons clés suivants :

4.1 Planning Prévisionnel

Semaines	Phase	Livrables et Activités
S1 – S2	Cadrage & Conception	Rédaction du Cahier des Charges, conception de l'architecture logicielle, création des templates de base.
S3 – S4	Développement Core (Backend)	Implémentation du moteur de compilation FreeMarker, génération de la structure de dossiers et du ZIP.
S5 – S6	Développement Client (Frontend)	Création du formulaire adaptatif sous Angular 18, intégration RxJS, connexion à l'API REST.
S7	Intégration DevOps & Tests	Génération dynamique de Dockerfiles/Jenkinsfiles, phase de tests unitaires et d'intégration.
S8	Déploiement & Clôture	Déploiement de la solution sur serveur local, rédaction du rapport de stage et maintenance de fin de projet.

TABLE 2 – Planning prévisionnel de réalisation

4.2 Critères d'Acceptation

Le projet sera considéré comme conforme et validé selon les indicateurs suivants :

- ✓ Le projet généré compile et démarre localement sans intervention manuelle du développeur.
- ✓ L'interface web de génération est réactive (temps de génération globale du package inférieur à 15 secondes).
- ✓ L'architecture respecte les principes SOLID et dispose d'une couverture de tests minimale de 70%.
- ✓ Les pipelines CI/CD (Jenkinsfile) et les descripteurs d'infrastructure (Docker Compose) générés s'exécutent avec succès sur un environnement vierge sans erreur de syntaxe.

5 Conclusion

Le projet **Webpick Initializr** représente un atout majeur pour l'amélioration continue des processus DevOps de l'agence Webpick. En s'appuyant sur un couplage robuste entre Angular 18 et Spring Boot 3, ce stage d'ingénierie offrira un livrable de qualité industrielle tout en permettant à l'étudiant de consolider ses compétences sur des architectures modernes de niveau professionnel.